



UNCUYO
UNIVERSIDAD
NACIONAL DE CUYO

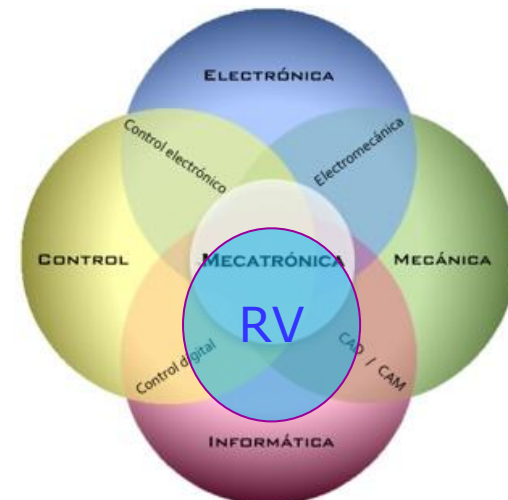


FACULTAD
DE INGENIERÍA

Realidad Virtual

Processing (parte 1)

Ingeniería en Mecatrónica



¿Qué es processing?

- Lenguaje derivado (dialecto) de Java, python, JScript y otros.
- Desarrollado por artistas y para artistas
- Orientado al arte gráfico, las animaciones y otras aplicaciones gráficas
- Processing es software libre, disponible en las plataformas donde el lenguaje base esté disponible
- Desarrollado a partir en el MIT por Casey Reas y Ben Fry. Inspirado en DBN (Design By Numbers) de John Maeda
- <https://processing.org>

¿Por qué Processing?

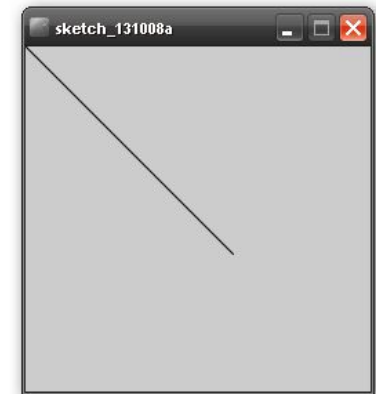
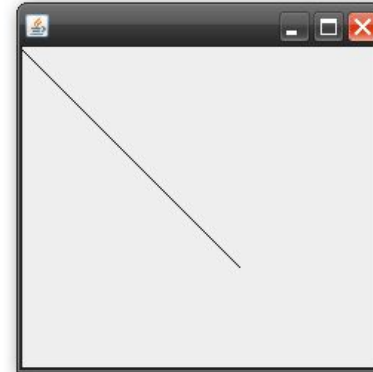
- Es muy fácil de aprender. Presenta una mejor curva de aprendizaje que OpenGL + GLUT u otras alternativas
- Potente. Permite desarrollar aplicaciones desde muy sencillas a muy complejas.
- Escalable. Se puede combinar programación en processing con Java puro y las librerías propias de Java. Una aplicación Java 'pura' también puede usar las librerías de processing.
- Dispone de un propio y sencillo IDE
- Genera aplicaciones listas para ser ejecutadas (incl.en [www](http://www.processing.org))
- Admite el desarrollo para dispositivos móviles
- Conexión con dispositivos terminados o de prototipado electrónico: proyectos Arduino y Wiring
- Flexible para iniciarse en el lenguaje ya que permite diferentes modos de programación

Processing vs. Java

- El preprocesador de processing permite usar una sintaxis más relajada
- Admite 3 modos de programación
 - básico: sin funciones propias, únicamente variables globales y directamente líneas de código
 - Procedural: definiendo nuestras propias funciones 'a la C', esto es sin tener que definir clases y métodos
 - OO: definir y usar clases (con una sintaxis más sencilla)
- La integración en clases de los modos primeros es transparente al programador

Ejemplo processing vs. java

```
3 import java.awt.EventQueue;
4 import java.awt.Graphics;
5 import javax.swing.JFrame;
6
7 public class soloLinea extends JFrame {
8
9     public static void main(String[] args) {
10         EventQueue.invokeLater(new Runnable() {
11             public void run() {
12                 try {
13                     soloLinea frame = new soloLinea();
14                     frame.setVisible(true);
15                 } catch (Exception e) {
16                     e.printStackTrace();
17                 }
18             }
19         });
20     }
21
22     public soloLinea() {
23         setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);
24         setBounds(100, 100, 250, 250);
25     }
26
27     public void paint (Graphics g)
28     {
29         super.paint(g);
30         g.drawLine (0, 28, 150, 178);
31     }
32 }
```

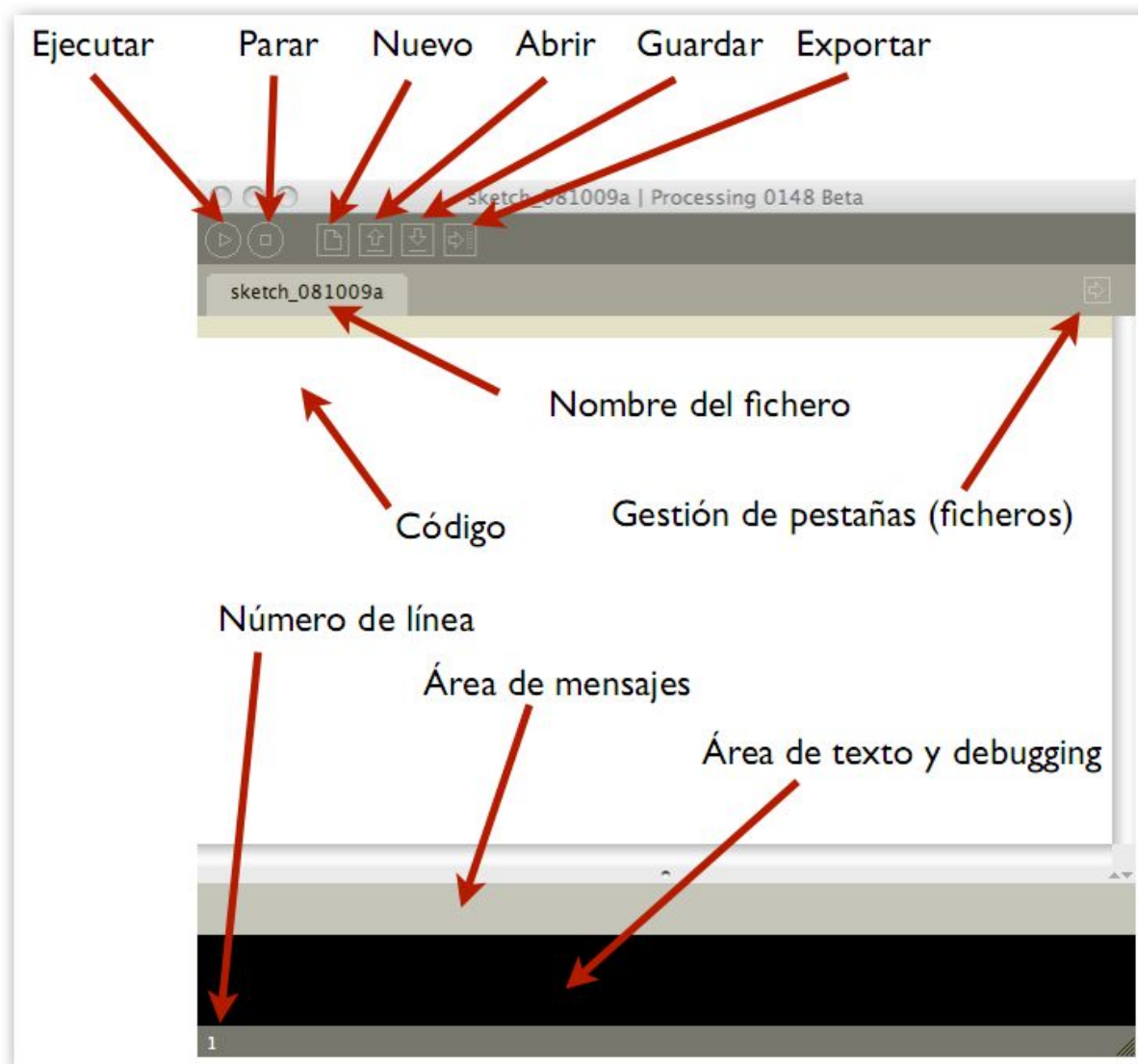


```
sketch_131008a
size(250,250);
line(0,0,150,150);
```

algunos paquetes y librerías gráficas

- 3D Core Graphics System.ACM y SIGGRAPH (1977)
- GKS (Graphical Kernel System). 2D. ISO-ANSI (1985)
- GKS-3D (1988)
- PHIGS (Programmer's Hierarchical Interactive Graphics (1988)
- SRGPY SPHIGS (Foley)
- OpenGL
- XWindows, Microsoft Windows, Mac OS ...
- VRML, X3D (orientados a internet)
- POV-Ray, Renderman
- Java2D,Java3D
- DirectX (Direct-3D)
- etc.

Processing IDE



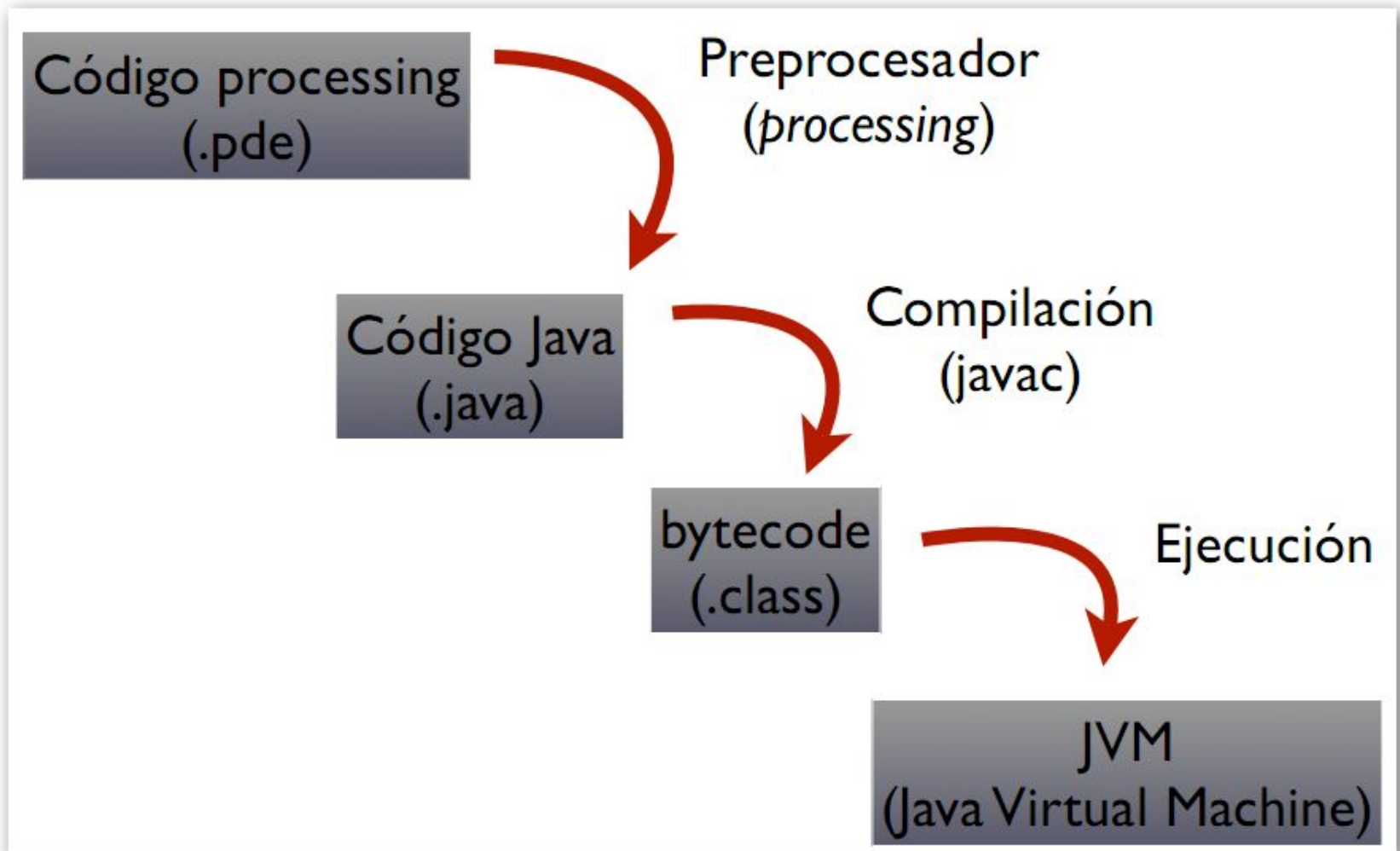
Generalidades de IDE y archivos

- Al guardar una aplicación, se crea un directorio con ese nombre y dentro un archivo con la extensión .pde
- Nombres de archivo: máximo 64 caracteres sin espacios
- El directorio mantiene la idea de directorio de proyecto. Dentro pueden haber más archivos (abiertos en el IDE como pestañas) ya sea con extensión .pde u otras (.java, de imagen, etc.)
- Los archivos adicionales corresponderán a nuevas funciones o clases.
- Si hay más de un archivo debe definirse en el archivo principal la función setup
- Para abrir un proyecto basta con abrir cualquier .pde del interior del directorio

Descripción de Opciones del IDE

- **Run y Stop**: inicia o detiene la ejecución de la aplicación
- **Export**: genera un 'applet' de la aplicación
- **Export Application**: genera una aplicación autónoma para Mac OS, Linux o Windows
- El directorio 'Sketchbook1' es el directorio por defecto para el usuario pero se puede almacenar las aplicaciones en cualquier directorio
- **Import Library**: facilita la inclusión de sentencias import para las librerías requeridas
- **Add File**: para añadir imágenes, fuentes u otra información multimedia a un directorio especial en nuestra carpeta de aplicación: la carpeta 'data'.
- **Create font**: creación de applets que utilicen fuentes de nuestro sistema
- **Help**: para consultar la información de processing

Lenguaje compilado/interpretado



Tipos de datos

- Variables

```
type name;
```

- Tipos básicos

```
byte, short, int(178), long (8864L), float (37.266F),  
double (37.266/26.77e3), char ('c'), boolean(true/false)
```

- Arrays

```
byte[ ] array;
```

- Inicialización

```
type name = value;
```

- Constantes

```
final type variable = value;
```

Operadores y precedencia

- Aritméticos: `+`, `-`, `*`, `/`, `%`, `++`, `--`
- Relacionales: `>`, `>=`, `<`, `<=`, `==`, `!=`
- Condicionales (logic): `&&`, `||`, `!`, `&`, `|`, `^`
- Operaciones con bits: `<<`, `>>`, `&`, `|`, `^`, `~`
- Asignación: `=`, `+=`, `-=`, `*=`, `/=`, `%=`, `&=`, `|=`, `^=`, `<<=`, `>>=`
- Otros: `?:`, `[]`, `.`, `(type)`, `new`, `instanceof`
- Precedencia:

1. `expr++`, `expr--`

2. `++expr`, `--expr`, `~`, `!`

3. `*`, `/`, `%`

4. `+`, `-`

5. `<<`, `>>`

6. `<`, `>`, `<=`, `>=`, `instanceof`

7. `==`, `!=`

8. `&`

9. `^`

10. `|`

11. `&&`

12. `||`

13. `?:`

14. `=`, `+=`, `-=`, `*=`, `/=`,
`%=`, `&=`, `^=`, `|=`, `<<=`,
`>>=`

Estructuras de control

- Delimitadores de bloque: { instrucciones; }
- while:

```
while ( expr ) { instrucciones }  
do { instrucciones } while ( expr )
```
- for

```
for ( begin; end; inc ) {instrucciones }
```
- if/else

```
if ( expr ) {instrucciones }  
if ( expr ) {instrucciones } else { instrucciones }
```
- switch

```
switch ( var ) { case val: instrucciones default: }
```
- Saltos: break, continue, return

Strings

- **Strings:**
 - `String mensaje = "hello";`
 - `String mensaje = "hello" + "bye";`
- **Gran cantidad de métodos:**
 - `length()`
 - **Acceso a un carácter:** `charAt()`
 - `substring()`
 - **Modificación:**
`concat(), replace(), trim(), toLowerCase(), etc.`
 - **Búsqueda:** `indexOf(), lastIndexOf(), etc.`
 - **Comparación:** `startsWith(), endsWith(), compareTo(), etc.`
 - `etc.`
- **Ejemplos:**

```
String mensaje = "hello" + "bye";
int len = mensaje.length();
int len = "hello".length();
```

Arrays

- Un array en processing (Java) es un objeto.
- Su longitud se obtiene mediante el atributo length

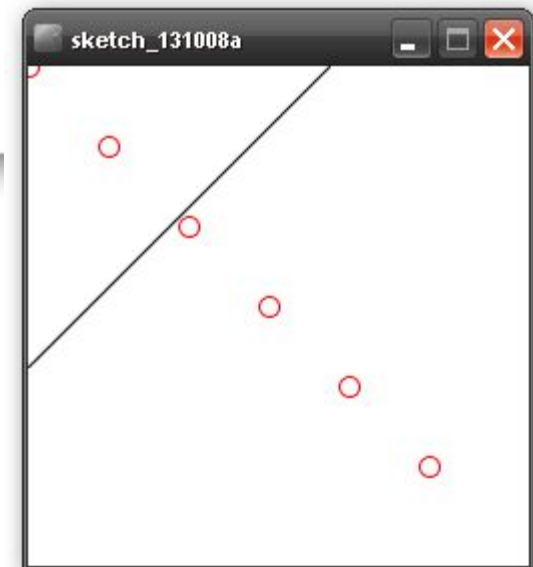
```
int[] vector; // vector es 'null'
vector = new int[3]; // creamos 3 componentes
int len = vector.length; // Longitud = 3
int item = vector[2]; // acceso a una componente

int [][] matriz = new int[4][4];
matriz[0][0] = 12; // Matrices
```

Ejemplo: Modo Básico

```
size(250,250);

int i;
background (255,255,255); // pantalla a blanco
stroke(0); // trazo de color negro
line(150,0,0,150);
stroke(255,0,0); // trazo de color rojo
for (i = 0; i <= 200; i+=40){
  point (i, i);
  ellipse(i,i, 10,10);
}
```



Ejemplo: Modo Estructurado

```
// Hay que implementar setup()
void setup(){
    size(100,100); // Tamaño de ventana de la aplicación
    background(0);
    noFill (); // Sin relleno de los círculos
    dibujaCirculoAleatorio();
}

void dibujaCirculoAleatorio(){
    int r;
    for(int i=0; i<100; i++){
        r= int(random(50)); // Color del trazo
        stroke (random(255), random(255), random(255));
        ellipse(random(100), random(100), r, r);
    }
}
```



Ejemplo: Modo Estructurado + draw()

```
// Hay que implementar setup()
void setup(){
    size(100,100); // Tamaño de ventana de la aplicación
    background(0);
    noFill (); // Sin relleno de los círculos
}

void draw(){
    dibujaCirculoAleatorio();
}

void dibujaCirculoAleatorio(){
    int r=int(random(50)); // Color del trazo
    stroke (random(255), random(255), random(255));
    ellipse(random(100), random(100), r, r);
}
```



Una forma de Obtener Animaciones

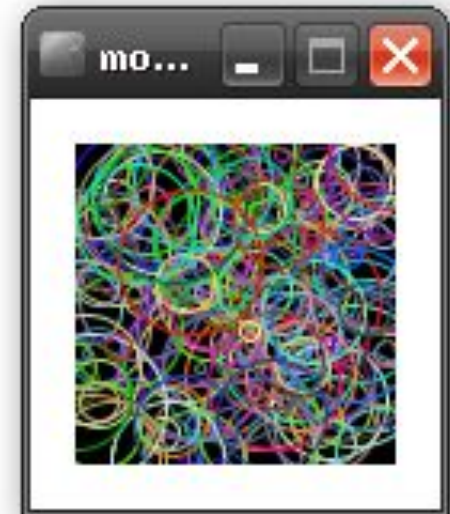
Ejemplo: Modo Orientado a Objetos

```
// Hay que implementar setup()
void setup(){
  size(100,100); // Tamaño de ventana de la aplicación
  background(0);
  noFill (); // Sin relleno de los círculos
}

void draw(){
  Circulo c = new Circulo ();
  c.dibujaCirculoAleatorio();
}

// Clase circulo
class Circulo{
  // Atributos
  int r = int (random(50));
  int x = int(random(100));
  int y = int(random(100));
  color c = color (random(255), random(255), random (255));

  void dibujaCirculoAleatorio(){
    stroke (c);
    ellipse(x, y, r, r);
  }
}
```



Modos de Visualización

- Processing dispone de varios modos de visualización: **JAVA2D**, **P2D**, **P3D** y **OPENGL**
- Hay otras posibilidades mediante el uso de librerías específicas (por ejemplo, renderizado basado en trazado de rayos)
- El modo de visualización se debe especificar como tercer argumento en la función **size()**
- El modo por defecto es JAVA2D

Color y Opciones de Trazado

- `stroke()`: fija el color de los trazos
 - `stroke (255)` => RGB(255, 255, 255) escala de grises
 - `stroke (128, 0, 128)` => Cualquier valor RGB
- `strokeWeight()`: fija el grosor de los trazos
 - `strokeWeight (5)` => Grosor 5
- `fill()`: Fija el color de relleno de figuras 2D
 - `fill(128)` => RGB(128, 128, 128) escala de grises
 - `fill(200, 120, 90)` => RGB(200, 120, 90)
- `background()`: Borra la ventana con el color especificado
 - `background (0)`
 - `background(128, 100, 128)`
- `noFill()`: Las figuras 2D se dibujaran sin relleno
- `noStroke()`: Las figuras 2D se dibujaran sin trazo

Primitivas básicas de processing

- `point(x, y)`: Dibuja un punto en la coordenadas (x, y) especificadas
- `set(x, y, color)`: Dibuja un punto en las coordenadas (x, y) con un color concreto
- `line(x1, y1, x2, y2)`: traza una línea recta entre los puntos (x1, y1) y (x2, y2)
- `strokeCap(tipo)`: donde tipo puede ser ROUND PROJECT o SQUARE
 - `strokeCap(PROJECT);`
 - `line(50, 75, 150, 75) ;`

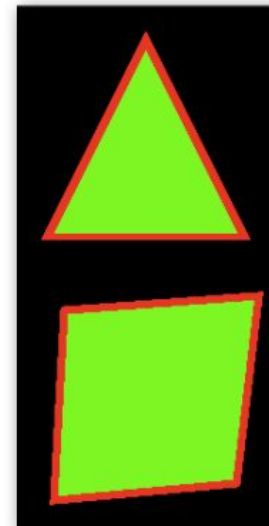


Elipses, círculos y arcos

- `ellipse(x, y, ancho, alto)`: traza una elipse en las coordenadas (x, y) de ancho y alto suministrados
- `ellipseMode()`: fija el modo de interpretación de los parámetros de la elipse
 - `ellipseMode (CENTER)` => (x, y) es el centro de la elipse (es el modo por defecto).
 - `ellipseMode (RADIUS)` => igual que el anterior, pero ancho y alto son radios y no diámetros
 - `ellipseMode (CORNER)` => (x,y) se refiere a la esquina superior izquierda del rectángulo envolvente de la elipse
 - `ellipseMode (CORNERS)` => los cuatro parámetros de la elipse hacen referencia a dos puntos opuestos del rectángulo envolvente de la elipse
- `arc(x, y, ancho, alto, inicio, fin)`: traza un arco como un sector de elipse en las coordenadas (x, y) con el ancho y alto suministrados. El ángulo es horario

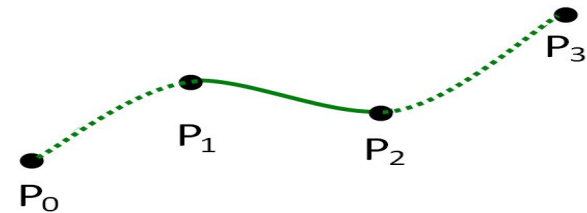
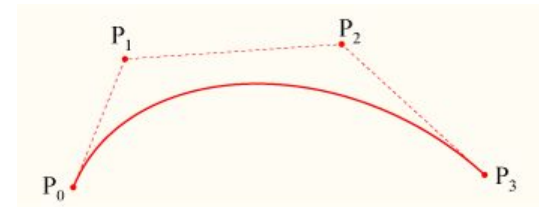
Rectángulos, triángulos y cuadriláteros

- `rect(x, y, ancho, alto)`: Dibuja un rectángulo
- `rectMode()`: fija el modo de interpretación de los parámetros del rectángulo. Como antes puede ser CENTER, RADIUS, CORNER y CORNERS
- `triangle(x1, y1, x2, y2, x3, y3)`: Dibuja un triángulo a partir de sus tres vértices
- `quad(x1, y1, x2, y2, x3, y3, x4, y4)`: Dibuja un cuadrilátero
- Ejemplo
 - `triangle(100, 25, 25, 175, 175, 175);`
 - `quad(38, 231, 186, 220, 169, 363, 30, 376);`



Curvas de Bezier y de Catmull-Rom

- `bezier(x1, y1, cx1, cy1, cx2, cy2, x2, y2)`
 - Son curvas cúbicas (un punto de inflexión)
 - Para su caracterización se precisa de 4 puntos: inicio y fin más 2 "atractores" que modifican la curvatura
- `curve(cx1, cy1, x1, y1, x2, y2, cx2, cy2)`
 - Útil para conseguir una curva que interpole una colección de puntos, especialmente shapes libres
 - Traza una curva de $(x1, y1)$ a $(x2, y2)$. Los puntos de control marcan la curvatura de inicio de la curva y de finalización de la misma



Imágenes

- **PImage**: Clase para crear objetos del tipo imagen
- **loadImage ()**: permite cargar una imagen en un objeto PImage en formato: GIF,JPG,TGA y PNG
 - Los archivos de las imágenes deben estar en el directorio **data** donde radica la aplicación (el PDE tiene la opción 'Add File' que permite seleccionar la imagen e insertarla en data)
- **image ()**: traza/visualizar la imagen
 - `image (imagen, x, y)` -> Muestra en (x, y) sin ajuste de tamaño
 - `image (imagen, x, y, ancho, alto)` -> Muestra en (x, y) reescalada al ancho y alto especificados
- **Ejemplo:**
 - `PImage foto = loadImage("foto.jpg");`
 - `image(foto, 0, 0);`
- **createImage(ancho, alto, modo de color)**: crea una nueva imagen con un modo de color que puede ser RGB, ARGB o ALPHA

Transformaciones 2D

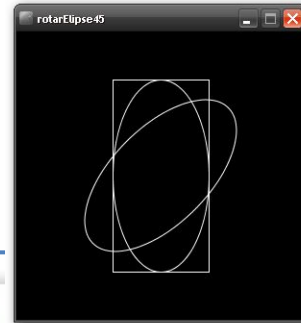
- Toda primitiva de dibujo está afectada por una matriz de transformación interna (matriz modelo/vista)
- Esta matriz permite transformar el sistema de coordenadas original del modelo en el sistema de coordenadas del dispositivo
- Inicialmente, la matriz modelo/vista es la matriz identidad, no afectando a ninguna primitiva de dibujo
- La definición de esta matriz se lleva a cabo mediante las funciones
 - `rotate(ángulo)`
 - `translate(tx, ty)`
 - `scale(sx, sy)`
- El orden en el que se deben codificar estas funciones corresponde al orden inverso en el que se aplicarán, es decir, el orden de aplicación de las mismas será de abajo a arriba

Otras Operaciones

- `resetMatrix ()`: Inicializa la matriz modelo/vista igualándola con la matrix identidad
- processing dispone de lo que se denomina una pila de matrices
- `pushMatrix ()`: provoca que la matriz de transformación modelo/vista actual sea apilada sin perder su actual valor
- `popMatrix ()`: provoca que la matriz de transformación modelo/ vista actual pierda su valor por el del tope de la pila de matrices. Este tope es efectivamente desapilado de la pila de matrices
- En una cadena de transformaciones se llamará a `pushMatrix()` cada paso intermedio al cual se quiera volver posteriormente llamando a `popMatrix ()`
- `pushMatrix ()` y `popMatrix ()` deben tener la misma cantidad de llamadas

Ejemplo con transformaciones

Rotar una ellipse 45°



```
// Un ejemplo de uso de
// pushMatrix() y popMatrix()
size(300, 300);

noFill();
background(0);
stroke(255);

// Un primer cambio de sistema de
// coordenadas:
// Movemos el (0,0) a la mitad
// de la ventana
translate(width / 2, height / 2);

// Dibujamos en este sistema
// una ellipse en la mitad de
// la pantalla
ellipse(0, 0, 100, 200);

// Almacenamos este primer
// sistema de coordenadas
pushMatrix();
```

```
// Un segundo sistema de
// coordenadas:
// Adicionalmente al cambio
// anterior,
// aplicamos una rotación
rotate(PI / 4.0);

// Dibujamos la misma ellipse,
// pero en este caso quedará
// afectada por las dos
// transformaciones
// anteriores
ellipse(0, 0, 100, 200);

// Recuperamos el primer sistema
// de coordenadas
popMatrix();

// Dibujamos de nuevo.
// Ahora estamos afectados
// únicamente
// por el primer translate
rectMode(CENTER);
rect(0, 0, 100, 200);
```

Bibliografía

- REAS, Casey y FRY, Ben (2007): "Processing. A programming handbook for visual designers and artists". The MIT Press Cambridge, England
- GREENBERG, Ira (2007): "Processing. Creative Coding and Computational Art", Ed. Friends of, Springer-Verlag, N.Y.
- Enlaces Web
 - <https://processing.org/>
 - <https://processing.org/tutorials/>
 - <https://processing.org/examples/>
 - <https://learningprocessing.com/examples/>
 - <https://www.openprocessing.org/>
 - <https://formandcode.com/code-examples/>